Java w IntelliJ IDEA - algorytmy i sortowanie

BLOK 9 - minimum, sortowanie bąbelkowe, wstawianie i złożoność

1. Co oznacza słowo algorytm?

Algorytm to dokładny opis kolejnych czynności prowadzących do rozwiązania określonego problemu. Nie musi być zapisany w języku programowania. Przepis kulinarny także zawiera kolejność działań, ale algorytm komputerowy powinien określać operacje i warunki w sposób na tyle precyzyjny, aby komputer mógł je wykonać. Dobry algorytm bierze pod uwagę również przypadki nietypowe, np. pustą tablicę albo kilka jednakowych wartości.

Załóżmy, że mamy listę ocen i chcemy znaleźć najwyższą. Możemy przejrzeć wszystkie wyniki i zapamiętywać największy dotychczas znaleziony. Nie musimy od razu sortować całej tablicy. Algorytm powinien wykonywać tylko tyle pracy, ile wymaga zadanie.

2. Szukanie minimum - krok po kroku

Aby znaleźć minimum w niepustej tablicy, przyjmujemy na początek pierwszy element jako aktualnie najmniejszy. Porównujemy z nim każdą kolejną liczbę. Jeżeli znajdziemy mniejszą, aktualizujemy minimum. Po odwiedzeniu wszystkich elementów otrzymujemy najmniejszą wartość.

```
int[] numbers = {8, 3, 6, 2};
int min = numbers[0];
for (int i = 1; i < numbers.length; i++) {
    if (numbers[i] < min) {
        min = numbers[i];
    }
}
System.out.println(min);
```

```
Wynik:
2
```

Przed pętlą `min = 8`. Po porównaniu z 3 minimum zmienia się na 3. Liczba 6 nie zmienia wyniku. Ostatnia liczba 2 jest mniejsza od 3, dlatego nowe minimum to 2. Pętla wykonuje trzy porównania dla czterech elementów. Dla tablicy o długości `n` potrzebujemy w tym wariancie `n - 1` porównań.

3. Minimum w wybranym wierszu tablicy 2D

Jeżeli dane są podzielone na wiersze i kolumny, musimy poprawnie wybrać, które elementy porównujemy. Dla minimum w jednym wierszu przechodzimy tylko po elementach tego wiersza. Dla minimum w kolumnie przechodzimy po kolejnych wierszach, utrzymując stały numer kolumny.

```
int[][] table = {{4, 7, 1}, {9, 2, 6}};
int row = 1;
int min = table[row][0];
for (int col = 1; col < table[row].length; col++) {
    if (table[row][col] < min) min = table[row][col];
}
System.out.println(min);
```

```
Wynik:
2
```

Wybieramy `row = 1`, czyli drugi wiersz `{9, 2, 6}`. Początkowe minimum wynosi 9. Po sprawdzeniu 2 zmienia się na 2, a 6 nie powoduje zmiany. Przy tablicach o różnej długości wierszy trzeba sprawdzać, czy wybrana kolumna rzeczywiście istnieje.

4. Sortowanie - jaki wynik chcemy otrzymać?

Sortowanie polega na ułożeniu danych zgodnie z określonym porządkiem, na przykład rosnąco albo malejąco. Jeżeli dane wejściowe to `{5, 2, 8, 1}`, sortowanie rosnące powinno dać `{1, 2, 5, 8}`. Wartości nie mogą zniknąć ani zostać zmienione - zmienia się jedynie kolejność elementów.

Przed napisaniem algorytmu należy zdecydować, czy sortujemy całą tablicę, czy tylko fragment, czy modyfikujemy oryginalną tablicę, czy tworzymy nową, oraz jak traktujemy wartości jednakowe. W pierwszych ćwiczeniach najczęściej sortujemy rosnąco liczby `int` w tej samej tablicy.

5. Sortowanie bąbelkowe - podstawowa zasada

Bubble sort, czyli sortowanie bąbelkowe, porównuje sąsiednie elementy. Jeżeli stoją w złej kolejności, zamienia je miejscami. Po każdym pełnym przejściu największy z pozostałych nieuporządkowanych elementów znajduje się na końcu rozpatrywanego fragmentu tablicy.

```
int[] a = {5, 2, 4};
for (int pass = 0; pass < a.length - 1; pass++) {
    for (int j = 0; j < a.length - 1 - pass; j++) {
        if (a[j] > a[j + 1]) {
            int temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
        }
    }
}
```

Pierwsze porównanie sprawdza 5 i 2. Ponieważ 5 jest większe od 2, zamieniamy elementy i otrzymujemy `{2, 5, 4}`. Następnie porównujemy 5 i 4, zamieniamy je i mamy `{2, 4, 5}`. Ostatni element jest już na swoim miejscu. W kolejnym przejściu sprawdzamy tylko początek tablicy. Wynik to `{2, 4, 5}`.

6. Dlaczego przy bubble sort potrzebne są dwie pętle?

Pętla wewnętrzna porównuje pary sąsiadów w jednym przebiegu. Pętla zewnętrzna powtarza przebiegi, ponieważ jedno przejście nie musi ustawić wszystkich elementów. Na przykład w `{4, 3, 2, 1}` po pierwszym przejściu liczba 4 dotrze na koniec, ale liczby 3, 2 i 1 nadal nie będą całkowicie uporządkowane.

Warunek `j < a.length - 1 - pass` zabezpiecza odwołanie `a[j + 1]` przed wyjściem poza tablicę. Końcowy fragment jest skracany, gdyż wartości ustawione w poprzednich przebiegach nie wymagają ponownego sprawdzania. Dla początkującego ważniejsze od zapamiętania nagłówków pętli jest zrozumienie tego, *dlaczego* druga pętla nie odwiedza już końca tablicy.

7. Usprawnienie bubble sort - kiedy można zakończyć wcześniej?

Jeśli w pełnym przejściu nie dokonamy żadnej zamiany, tablica jest już uporządkowana i nie trzeba kontynuować. Możemy wykorzystać pomocniczą zmienną logiczną, która pamięta, czy podczas przebiegu coś zostało przestawione.

```
boolean swapped = false;
for (int j = 0; j < a.length - 1; j++) {
    if (a[j] > a[j + 1]) {
        int temp = a[j];
        a[j] = a[j + 1];
        a[j + 1] = temp;
        swapped = true;
    }
}
```

Jeżeli po przebiegu `swapped` nadal wynosi `false`, żadne dwa sąsiednie elementy nie były w złej kolejności. W poprawnie skonstruowanej zewnętrznej pętli pozwala to zakończyć sortowanie wcześniej. Jeżeli `swapped` wynosi `true`, nie mamy jeszcze gwarancji, że cała tablica jest posortowana.

8. Sortowanie przez wstawianie - budowanie posortowanego fragmentu

Insertion sort, czyli sortowanie przez wstawianie, przypomina wkładanie kolejnych kart w odpowiednie miejsce w dłoni. Traktujemy pierwszy element jako już posortowany. Następnie pobieramy kolejny i przesuwamy większe elementy w prawo, aż znajdziemy odpowiednie miejsce dla wybranej wartości.

```
int[] a = {5, 2, 4};
for (int i = 1; i < a.length; i++) {
    int key = a[i];
    int j = i - 1;
    while (j >= 0 && a[j] > key) {
        a[j + 1] = a[j];
        j--;
    }
    a[j + 1] = key;
}
```

Przy pierwszej iteracji `key = 2`. Liczba 5 jest większa od 2, dlatego przesuwamy ją w prawo i zapisujemy 2 na początku. Tablica staje się `{2, 5, 4}`. Przy następnej iteracji `key = 4`. Przesuwamy 5 w prawo i wstawiamy 4 przed nią. Ostatecznie dostajemy `{2, 4, 5}`.

9. Dlaczego warunek w insertion sort ma dwie części?

W pętli `while (j >= 0 && a[j] > key)` sprawdzamy najpierw, czy indeks jest prawidłowy. Dopiero potem odczytujemy element tablicy. Ma to duże znaczenie, gdy przesuwamy element na początek tablicy i licznik `j` może spaść do -1. Dzięki skróconemu obliczaniu `&&` prawa strona nie jest sprawdzana, jeśli `j >= 0` okazało się fałszywe.

Jeżeli odwrócilibyśmy kolejność na `a[j] > key && j >= 0`, program mógłby spróbować odczytać element o indeksie -1 i zgłosić wyjątek. Zapis warunku to zatem część poprawności algorytmu, nie tylko szczegół stylistyczny.

10. Sortowanie malejące

W obu algorytmach kierunek sortowania zależy od sposobu porównywania wartości. Przy sortowaniu rosnącym większy element przesuwamy dalej w prawo. Przy sortowaniu malejącym chcemy, aby większe wartości znalazły się na początku. W prostym wariancie bubble sort wystarczy odpowiednio zmienić warunek porównania sąsiednich elementów.

```
if (a[j] < a[j + 1]) {
    int temp = a[j];
    a[j] = a[j + 1];
    a[j + 1] = temp;
}
```

Jeżeli lewy element jest mniejszy od prawego, zamieniamy je, aby większy przesunął się w lewo. To działa jako część pełnego algorytmu sortowania. Sama pojedyncza instrukcja `if` nie posortuje całej tablicy; musi znaleźć się w odpowiednich pętlach.

11. Złożoność czasowa - od czego zależy liczba operacji?

Wprowadzenie do złożoności algorytmów pomaga rozumieć, dlaczego dwa programy dają ten sam wynik, ale nie są równie wydajne. Dla tablicy `n` elementów liniowe szukanie minimum wymaga liczby porównań proporcjonalnej do `n`, co zapisujemy jako **O(n)**. Klasyczne sortowania bubble sort i insertion sort mają w najgorszym przypadku liczbę operacji proporcjonalną do n^2, co zapisujemy jako **O(n²)**.

Przy podwojeniu liczby elementów algorytm liniowy potrzebuje mniej więcej dwa razy więcej porównań, a kwadratowy w typowych najgorszych przypadkach około cztery razy więcej. Notacja O opisuje tempo wzrostu, a nie dokładną liczbę sekund. Czas wykonania zależy także od komputera, środowiska i danych wejściowych.

Algorytm	Przykładowa złożoność czasu	Sens

szukanie minimum	`O(n)`	**przegląd elementów po kolei**
bubble sort	`O(n²)` w najgorszym przypadku	wielokrotne przejścia po tablicy
insertion sort	`O(n²)` w najgorszym przypadku	przesuwanie elementów przy wstawianiu

12. Złożoność pamięciowa i stabilność sortowania

Złożoność pamięciowa mówi, ile dodatkowej pamięci potrzebuje algorytm wraz ze wzrostem danych. Klasyczne bubble sort i insertion sort wykonywane na istniejącej tablicy mogą działać z dodatkową pamięcią **O(1)**, ponieważ potrzebują tylko kilku pomocniczych zmiennych niezależnie od długości tablicy.

Stabilność sortowania dotyczy elementów o jednakowym kluczu. Stabilny algorytm zachowuje ich wzajemną kolejność. To ważne przy sortowaniu np. listy osób po ocenie, gdy dwie osoby mają taki sam wynik i chcemy zachować ich pierwotną kolejność. Typowe warianty bubble sort i insertion sort z porównaniem ścisłym mogą być stabilne.

13. Jak testować algorytm sortowania?

Wynik jednego testu nie wystarcza. Sprawdź przynajmniej tablicę już posortowaną, posortowaną odwrotnie, zawierającą duplikaty, z jednym elementem i pustą. Warto też przetestować liczby ujemne. Przy każdym teście sprawdź dwie właściwości: wartości są w oczekiwanej kolejności i żaden element nie został utracony ani dodany.

```
int[] a = {3, 3, -1, 2};
java.util.Arrays.sort(a);
System.out.println(java.util.Arrays.toString(a));
```

```
Wynik:
[-1, 2, 3, 3]
```

`Arrays.sort` jest gotową metodą biblioteczną. Można użyć jej do porównania wyniku własnego algorytmu, ale samo wywołanie tej metody nie zastępuje wykonania ćwiczenia polegającego na samodzielnym zaimplementowaniu bubble sort lub insertion sort.

Przykład 1. Example01BubbleSort

Sortujemy przez wielokrotne porównywanie sąsiednich elementów i ich zamianę.

```
package block9;

import java.util.Arrays;

public class Example01BubbleSort {
    public static void main(String[] args) {
        int[] a = {5, 1, 4, 2, 8};

        for (int end = a.length - 1; end > 0; end--) {
            boolean swapped = false;
            for (int i = 0; i < end; i++) {
                if (a[i] > a[i + 1]) {
                    int temp = a[i];
                    a[i] = a[i + 1];
                    a[i + 1] = temp;
                    swapped = true;
                }
            }
            if (!swapped) break;
        }

        System.out.println(Arrays.toString(a));
    }
}
```

Wynik:

```
[1, 2, 4, 5, 8]
```

- Instrukcja `int[] a = {5, 1, 4, 2, 8};` deklaruje tablicę. Po utworzeniu odwołujemy się do jej elementów przez indeksy zaczynające się od zera.
- Nagłówek `for (int end = a.length - 1; end > 0; end--) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.
- Zapis `boolean swapped = false;` tworzy zmienną `swapped` typu `boolean` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Nagłówek `for (int i = 0; i < end; i++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.
- W zapisie `if (a[i] > a[i + 1]) {` program podejmuje decyzję na podstawie wartości logicznej. Instrukcje w bloku są wykonywane tylko wtedy, gdy wynik warunku jest `true`.
- Zapis `int temp = a[i];` tworzy zmienną `temp` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.

Przykład 2. Example02InsertionSort

Wstawiamy kolejne elementy w odpowiednie miejsce już posortowanego fragmentu.

```
package block9;

import java.util.Arrays;

public class Example02InsertionSort {
    public static void main(String[] args) {
        int[] a = {7, 3, 5, 2, 9, 1};

        for (int i = 1; i < a.length; i++) {
            int key = a[i];
            int j = i - 1;

            while (j >= 0 && a[j] > key) {
                a[j + 1] = a[j];
                j--;
            }
            a[j + 1] = key;
        }

        System.out.println(Arrays.toString(a));
    }
}
```

Wynik:

```
[1, 2, 3, 5, 7, 9]
```

- Instrukcja `int[] a = {7, 3, 5, 2, 9, 1};` deklaruje tablicę. Po utworzeniu odwołujemy się do jej elementów przez indeksy zaczynające się od zera.
- Nagłówek `for (int i = 1; i < a.length; i++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.
- Zapis `int key = a[i];` tworzy zmienną `key` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `int j = i - 1;` tworzy zmienną `j` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Warunek `while (j >= 0 && a[j] > key) {` jest sprawdzany przy kolejnych przejściach pętli. Jeśli przestanie być prawdziwy, następna iteracja nie zostanie wykonana.
- Instrukcja `j--;` zmienia wartość istniejącej zmiennej. Jej nowa wartość będzie użyta przy następnym sprawdzeniu warunku lub obliczeniu.

Przykład 3. Example03MinInRowColumn

Wyszukujemy najmniejszą wartość w wybranym wierszu lub kolumnie macierzy.

```
package block9;

public class Example03MinInRowColumn {
    public static void main(String[] args) {
        int[][] matrix = {
                {8, 3, 7},
                {4, 9, 1},
                {6, 2, 5}
        };

        int selectedRow = 1;
        int minRow = matrix[selectedRow][0];
        for (int col = 1; col < matrix[selectedRow].length; col++) {
            if (matrix[selectedRow][col] < minRow) minRow = matrix[selectedRow][col];
        }

        int selectedCol = 1;
        int minCol = matrix[0][selectedCol];
        for (int row = 1; row < matrix.length; row++) {
            if (matrix[row][selectedCol] < minCol) minCol = matrix[row][selectedCol];
        }

        System.out.println("Min w wierszu: " + minRow);
        System.out.println("Min w kolumnie: " + minCol);
    }
}
```

Wynik:

```
Min w wierszu: 1
Min w kolumnie: 2
```

- Zapis `int selectedRow = 1;` tworzy zmienną `selectedRow` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `int minRow = matrix[selectedRow][0];` tworzy zmienną `minRow` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Nagłówek `for (int col = 1; col < matrix[selectedRow].length; col++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.
- W zapisie `if (matrix[selectedRow][col] < minRow) minRow = matrix[selectedRow][col];` program podejmuje decyzję na podstawie wartości logicznej. Instrukcje w bloku są wykonywane tylko wtedy, gdy wynik warunku jest `true`.
- Zapis `int selectedCol = 1;` tworzy zmienną `selectedCol` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `int minCol = matrix[0][selectedCol];` tworzy zmienną `minCol` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.

Przykład 4. Example04Complexity

Liczymy kroki jednej pętli i dwóch pętli zagnieżdżonych. To konkretny sposób zrozumienia różnicy O(n) i O(n²).

```
package block9;

public class Example04Complexity {
    public static void main(String[] args) {
        int n = 100;
        long linearSteps = 0;
        long quadraticSteps = 0;

        for (int i = 0; i < n; i++) {
            linearSteps++;
        }

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                quadraticSteps++;
            }
        }

        System.out.println("O(n) kroki: " + linearSteps);
        System.out.println("O(n^2) kroki: " + quadraticSteps);
    }
}
```

Wynik:

```
O(n) kroki: 100
O(n^2) kroki: 10000
```

- Zapis `int n = 100;` tworzy zmienną `n` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `long linearSteps = 0;` tworzy zmienną `linearSteps` typu `long` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `long quadraticSteps = 0;` tworzy zmienną `quadraticSteps` typu `long` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Nagłówek `for (int i = 0; i < n; i++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.
- Instrukcja `linearSteps++;` zmienia wartość istniejącej zmiennej. Jej nowa wartość będzie użyta przy następnym sprawdzeniu warunku lub obliczeniu.
- Nagłówek `for (int j = 0; j < n; j++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.

Zadanie 5. FindMinColumn

W każdej kolumnie matrix znajdź najmniejszą wartość i ją wypisz. Dla tej macierzy minima kolejnych kolumn to 5, 1, 2.

```
package block9;

public class FindMinColumn {
    public static void main(String[] args) {
        int[][] matrix = {{5, 9, 2}, {8, 1, 7}, {6, 4, 3}};
        // TODO
    }
}
```

Zadanie 6. SortDescending

Posortuj a malejąco algorytmem bąbelkowym, bez Arrays.sort(). Wypisz tablicę. Oczekiwany wynik: 8, 5, 3, 2, 1.

```
package block9;

public class SortDescending {
    public static void main(String[] args) {
        int[] a = {3, 8, 1, 5, 2};
        // TODO
    }
}
```

- Instrukcja `int[] a = {3, 8, 1, 5, 2};` deklaruje tablicę. Po utworzeniu odwołujemy się do jej elementów przez indeksy zaczynające się od zera.

Zadanie 7. CompareSorts

W klasie `CompareSorts` należy samodzielnie uzupełnić algorytm zgodnie z komentarzem w pliku. Poniżej widoczny jest kod początkowy, do którego wpisujemy rozwiązanie.

Polecenie: Policz porównania w bubble sort i insertion sort dla trzech tablic: posortowanej rosnąco, malejąco i w losowej kolejności.

```
package block9;

public class CompareSorts {
    public static void main(String[] args) {
        // TODO
    }
}
```

14. Zadania - algorytmy i ich analiza

Zadanie 8. Znajdź minimum i maksimum w tablicy `{9, 4, -3, 7, 4}`. Zapisz osobno wartości przechowywane przez zmienną minimum po kolejnych porównaniach.

Zadanie 9. Ręcznie wykonaj dwa przebiegi bubble sort dla `{6, 3, 5, 1}`. Po każdym porównaniu zapisz aktualny stan tablicy. Wyjaśnij, który element jest już na właściwym miejscu po pierwszym przebiegu.

Zadanie 10. Zaimplementuj insertion sort dla `{7, 2, 6, 3}`. Po każdym ustawieniu `key` w poprawnym miejscu wyświetl cały stan tablicy.

Zadanie 11. Napisz algorytm znajdujący najmniejszą wartość w wybranej kolumnie macierzy. Sprawdź, co się stanie, jeśli nie wszystkie wiersze mają taką kolumnę.

Zadanie 12. Oszacuj, jak zmieni się liczba porównań dla 10 i 100 elementów w algorytmie liniowym oraz kwadratowym. Nie podawaj tego jako dokładnego czasu w sekundach.